

Early Detection of Crypto-Ransomware Using Network Traffic Analysis and Machine Learning Techniques

Phu Nguyen

Supervisor: Dr. Saqib Hakak

CS4997 Honours Thesis

Faculty of Computer Science

University of New Brunswick

ABSTRACT

Crypto-ransomware that reaches a corporate file server can encrypt the documents of an entire department from a single infected workstation. Research on detecting this type of attack has developed along two independent lines: host-based systems that monitor the endpoint filesystem, and network-based systems that monitor file-sharing traffic from an off-path probe. Each line reports high accuracy within its own observation channel, and the two are rarely evaluated against one another. The channels nevertheless overlap, because an operation issued against a mapped network drive traverses the same filesystem interface as an operation against local storage.

This thesis investigates whether the network channel is therefore redundant. A setup was constructed in which every execution, malicious and benign, yields both a host API trace and a capture of its file-sharing traffic, so that the two records of the same activity can be compared directly. Evaluation withholds one ransomware family at a time, which means that every reported detection is measured on a family the model has not observed during training. Across 109 ransomware executions spanning ten families and 187 benign samples, every configuration detects every execution, and the false-positive rates differ by a small number of benign runs. The alarm triggers 6 seconds after process launch, before any file on the share has been modified in 62 of 109 executions.

A second matter is the durability of the network channel. Enterprise file-sharing is migrating to SMB3 with encryption enabled, and many published network-based detectors rely on protocol metadata that encryption conceals. This work measures what survives by re-executing samples against an encrypted share. Command identification is almost entirely eliminated, whereas the packet-length signatures shift by 52-byte offset and byte volume is unaffected. A detector built

from the ten surviving features still detects every execution, at a false-positive rate of 4.3% against 2.1% on the plaintext.

TABLE OF CONTENTS

I. INTRODUCTION

A. Motivation

B. Research Questions

II. RELATED WORK

A. Host-Based Detection

B. Network-Based and Encrypted-Traffic Detection

III. METHODOLOGY

A. Laboratory Environment

B. Data Collection

C. Feature Extraction

1) Host Features:

2) Network Features:

D. Window Formulation

E. Model Training and Evaluation

F. Limitations of the Design

1) The Benign Corpus:

2) Instrumentation and Environment:

IV. EXPERIMENTAL RESULTS

A. RQ1: Contribution of the Network Channel

B. Feature Importance

C. RQ2: Detection Latency and Data Loss

D. RQ3: The Encrypted Protocol

1) Portability Across Server Implementations:

2) Behaviour Under Encryption:

3) Cost of the Loss:

V. CONCLUSION

VI. FUTURE WORK

References

I. INTRODUCTION

A. Motivation

Ransomware is the leading category of cyber incident affecting organisations. The Verizon Data Breach Investigations Report for 2025 identifies ransomware in 44% of confirmed breaches [1].

The mechanism is simple. The attacker encrypts the victim's files and withholds the decryption key. The operational damage then depends on how much data one compromised machine can reach, and how fast that data can be encrypted. Arányi et al. measured LockBit encrypting 53.8 GB across 100,000 files in four minutes and nine seconds [2]. Under those conditions, a detector that responds after one minute has already allowed the loss of most of a department's documents.

In a corporate environment the reachable data sits in two kinds of storage. A workstation holds system files and a user profile. The documents needed for business operations sit on a shared server, which the workstation mounts as a drive letter. One infected machine can therefore encrypt the working set of an entire department [3]. Most families target both kinds of storage.

This property makes file-sharing traffic a useful observation point. A probe placed on the network segment between the clients and the file server observes the aggregate effect of an infection. It needs no software on any endpoint. It also covers machines where an agent cannot be installed. It continues to operate when a sample disables the monitoring that an agent depends on.

The host channel observes which cryptographic primitives are invoked, the entropy of each buffer written, and the sequence of operations applied to each file. It observes these for the mapped drive as well as for local storage, because both are reached through the same interface. However, encryption on local storage generates no file-sharing traffic.

B. Research Questions

Host features are derived from the API trace and cover both storage locations. Network features are derived from the packet capture and cover the share alone. The following three questions structure the rest of this thesis.

RQ1. For ransomware that encrypts on a file server, does combining network telemetry with host telemetry improve detection, reduce false alarms, both, or neither?

RQ2. How much of the file server is lost by the time an alarm is raised, and how does that loss trade off against the number of consecutive windows required to confirm it?

RQ3. Which network features remain computable when the file-sharing protocol is encrypted, and what is the cost of losing the remainder?

The remainder of the thesis is organised as follows. Section II reviews the two lines of ransomware detection research compared. Section III describes the methodology. It covers the laboratory environment, the ransomware and benign samples, feature extraction, the window formulation, the evaluation protocol, and the limitations of the design. Section IV presents the experimental results and addresses each research question in turn. Section V concludes, and Section VI outlines directions for future work.

II. RELATED WORK

A. Host-Based Detection

CryptoDrop alarms only when file-type change, similarity, and entropy all appear together [5]. Redemption scores each process on its access pattern behind an I/O buffer [6]. RWGuard adds decoy files and hooks the CryptoAPI in memory, detecting the first malicious process in 3.45 seconds and all of them in 8.87 [7]. PayBreak instead keeps a copy of each key so the files can be recovered later [8]. Surveys of the field group these systems by the evidence each one consumes, whether API and system calls, I/O, or file-system activity [9], [10], and Vehabovic et al. treat host-based and network-based detection as separate categories rather than as complementary channels [11].

Others target the pre-encryption interval. Kok et al. classify samples from Windows API occurrence alone, using the 232 API features of the RISS dataset, and argue such a classifier could run at the pre-encryption stage [12]. Alqahtani and Sheldon warn that packers issue such calls too, and correlate them with I/O request packets [13]. Two weaknesses persist across this line of work. Entropy falls under Base64 encoding and partial encryption [14] and cannot separate compressed from encrypted files [15], and a memory-mapped write leaves no write call at all [16].

B. Network-Based and Encrypted-Traffic Detection

A wire probe needs no endpoint software but sees only SMB messages and their sizes. Morato et al. establish the approach with REDFISH, deciding in under 20 seconds across 19 families and restoring files from the capture itself [3]. Berrueta et al. reduce it to bytes read, bytes written, and

data-free commands per connection. Their false-positive validation uses 2,477 hours of campus traffic, detection averages 30.2 s for binaries seen in training, and at their recommended 30-second window unseen binaries encrypt about 113 MB before detection [17]. They also build for SMB3 from packet times, sizes and direction alone, reporting that the loss of protocol metadata raises false positives [17]. Hatungimana et al. identify SMB operations by frame length alone, publishing 7 lengths that a probe can recognise [18]. Work on encrypted traffic in other domains reaches the same conclusion, that packet size, timing and direction remain informative once the payload cannot be inspected [20]. An open repository of ransomware executions exists so that tools can be measured on common data [21], but published host-based and network-based results still rest on separate corpora.

III. METHODOLOGY

The methodology compares two observation channels on the same executions. Fig. 1 gives an overview of the design.

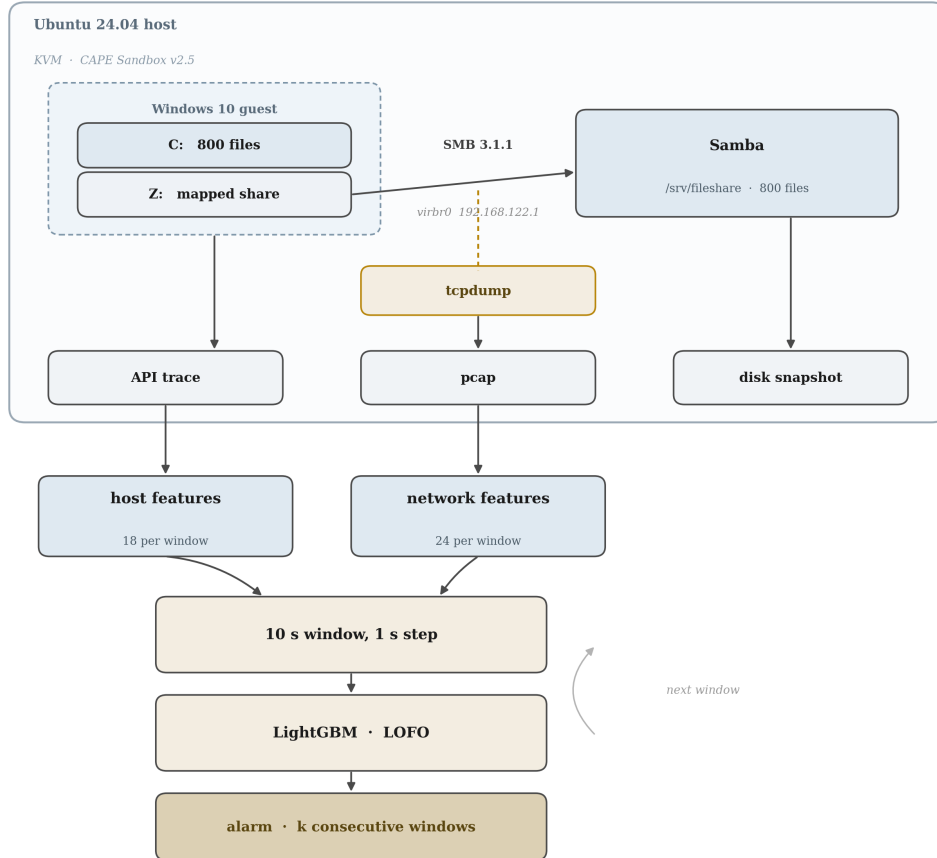


Fig. 1. Overview of the methodology.

A. Laboratory Environment

The laboratory reproduces the smallest segment of an enterprise network in which both observation channels can observe the same activity. It holds a workstation and the file server that workstation uses. The workstation stores local documents and mounts a departmental share, and the server exports that share. This matches the scenario in which an employee workstation is compromised and the ransomware encrypts both the files stored locally and those reachable

across the network. The same activity is therefore recorded twice: once as API calls at the endpoint, and once as SMB traffic on the wire.

Analysis runs on a bare-metal Ubuntu 24.04 host. The host executes CAPE Sandbox v2.5 over KVM, and the guest is Windows 10 x64. The guest is reverted to a clean snapshot before every detonation. The file server is a Samba instance on the same host. It exports a directory that the guest mounts as drive Z across the virtual bridge.

Both storage locations are seeded from the NapierOne corpus [22]. The share holds 800 files in a directory tree six to seven levels deep. The local disk holds a second set of 800 files, drawn from a different portion of the same collection. A copy of the share is kept outside the share itself and restored after each detonation. The restoration is verified by file count before the next sample is submitted.

The analysis timeout is 300 seconds. That value sits comfortably above the two-minute window which Kuchler et al. observe 98% of the basic blocks a sample will execute [23]. The negotiated protocol version is SMB 3.1.1 with encryption disabled, which is the default between a Windows 10 client and a Samba server. Two samples were also executed against a share that requires SMB3 encryption. These two encrypted runs measure the effect on the network features directly.

B. Data Collection

The corpus is assembled from two categories of executable, collected as follows.

- **Ransomware samples:** taken from the set of hashes published with the MLRan dataset[24]. Its authors verified the samples against VirusTotal. Samples for families that MLRan covers sparsely were collected directly from MalwareBazaar and tria.ge. The selection spans both the crypto era and the ransomware-as-a-service era.

- **Benign samples:** built for this experiment from signed Windows utilities which are 7za, robocopy, certutil, compact and PowerShell. Building the workloads rather than collecting them avoids the provenance problem Botacin et al. describe, in which benign samples gathered from download sites carry no guarantee of being benign [25].

TABLE I
RANSOMWARE FAMILIES RETAINED FOR TRAINING AND EVALUATION

Family	Executions	Family	Executions
MedusaLocker	14	Akira	11
Dharma	14	Maze	10
REvil	13	Conti	8
RagnarLocker	13	7ev3n	7
Nefilim	12	AvosLocker	7
Total			109

TABLE II
COMPOSITION OF THE BENIGN CORPUS

Behaviour	Tool	Action	What it does
Rewrite content in place	7za	Encrypt and overwrite	Encrypts a file with AES-256 and moves over the original
	PowerShell	Compress and overwrite	Gzip-compresses a file in memory and writes the result back
	PowerShell	Rewrite bytes	Reads a file and writes it back
	copy, move	Rewrite via temporary	Copies a file to a new name and moves it back over the original
Package a tree into an archive	7za	Archive with password	Builds one AES-256 archive from a directory tree, originals intact
	7za	Archive without password	The same in .7z or .zip with no encryption
	7za	Extract	Unpacks an archive, creating many files at once
Move or copy between locations	robocopy	Copy onto the share	Copies the local data to the share, leaving the source in place
	robocopy /MOV	Move onto the share	Copies to the share and deletes each local source
	robocopy /MOV	Move off the share	Pulls a subtree to local storage, deleting each file on the share

Behaviour	Tool	Action	What it does
	robocopy /MOV	Move within the share	Relocates one folder to another on the same share and back
Rename or re-flag without reading	ren	Append an extension	Renames every matching file to end in different extensions
	compact	Set NTFS compression	Compresses a tree on disk, leaving names and API-visible sizes unchanged
Read without writing	certutil	Hash every file	Computes a hash digest of each file, one variant indexing the tree first
Concurrent workloads	several	Run three or four at once	Starts copies, renames and reads in parallel on different subtrees

C. Feature Extraction

The feature extraction stage derives 42 quantities per observation window. They are computed from the two artifacts each execution produces.

- **Host analysis:** 18 features derived from the API call trace. They cover directory traversal, file operations, buffer entropy, cryptographic calls, process enumeration and thread creation. They observe activity on both local storage and the mapped drive, because both are reached through the same filesystem interface.

- **Network analysis:** 24 features derived from the packet capture. They cover SMB command rates, byte volume in each direction, and the packet-length identifiers that remain computable when the protocol layer is encrypted. They observe activity on the share alone.

A `w_` prefix denotes a rate measured within the current window. A `c_` prefix denotes a cumulative counter. Host counters accumulate from process start. Network counters accumulate from the point when the client opened its connection to the file server.

1) Host Features:

A ransomware family must perform a limited set of operations, and it performs them in a fixed order. Each stage leaves a trace at the API. The 18 host features follow that order.

1) Locating targets. The directory layout is unknown to the sample. It must therefore walk the tree before it can encrypt anything. `w_traversal_rate` counts that walking within the window, and `c_traversal` counts it across the whole execution.

2) Clearing obstacles. A file held open by another process cannot be encrypted. The sample enumerates the process table to find those processes. It also writes to the registry for persistence and configuration. `c_handle_enum` and `c_reg_write` accumulate these across the execution, and `w_reg_rate` measures the registry activity within the window.

3) Generating keys. No file can be encrypted before key material exists, so `c_keygen` is the earliest signal available. Kok et al. detect a sample during its preparation stage from the Windows API calls it issues, using API occurrence alone as the feature set [12]. RWGuard hooks the CryptoAPI functions in memory [7]. Alqahtani and Sheldon warn that a cryptographic call is not by itself evidence of an attack, because packers and legitimate software make the same calls [13].

4) The encryption loop. Ahmed et al. decompose this loop into four steps: access a file, read its contents, write the encrypted result, and dispose of the original [16]. They also report four patterns by which families combine the last two steps. Table III lists them.

TABLE III
THE FOUR FILE I/O PATTERNS REPORTED BY AHMED ET AL. [16]

Pattern	What the family does
Overwrite in place	Writes ciphertext over the original, then renames
Rename first	Renames the file, then writes ciphertext into it

File-to-file	Creates a new encrypted file, deletes the original
File-to-file with rename	Creates a new encrypted file, renames it, then deletes the original

Opens and writes are counted by `w_open_rate` and `w_write_rate`. Neither separates the classes on its own, because a copy utility also opens and writes at fast rates. The noisiest benign run in this corpus reaches 373 operations per second against a ransomware median of 407.

What distinguishes encryption is that the file read and the file written are the same file. A copy reads file A and writes file B. Encryption reads file A and writes file A. `w_rbo_rate` counts this. It records which files have been read, then fires when one of those same files is written, renamed or deleted. A copy never satisfies it.

`w_write_buf_entropy` measures the content of those writes. It takes the Shannon entropy of the buffer passed to each call. This is because a sample using intermittent encryption transforms only part of each file and leaves the rest as plaintext. A detector reading the file afterwards therefore sees a mixture, and the entropy of that mixture is far below ciphertext. The write buffer contains no such mixture. The bytes handed to the call are ciphertext however much of the file is left alone.

High entropy alone is ambiguous, because a process copying a compressed archive also writes high-entropy bytes. `w_iobuf_entropy_delta` resolves this. It subtracts the mean entropy of what the process read from the mean entropy of what it wrote. A copy consumes and produces the same bytes, so it sits near zero however high its entropy. Encryption consumes low-entropy documents and produces high-entropy ciphertext, so the difference is large.

There are ransomware families that can bypass the loop entirely. It creates a section, maps a view, transforms the bytes with CPU instructions, and unmaps. It issues no write call at any

point, so no buffer is available to inspect. Ahmed et al. name this case in their own limitations as one their kernel-level system may not detect [16]. `w_mmap_rate` observes it.

5) Disposing of the original. The four patterns require two forms of disposal, and each is counted separately. `w_rename_rate` fires when a target filename is the source filename with an element appended. `w_delete_rate` covers families that write a new file and remove the original. It counts files that flagged for deletion as well as direct deletions.

6) Organisation and normalisation. `c_thread` counts thread creations, which is how families encrypt several files at once. `c_injection` is set when the sample writes memory into another process, allocates memory there, or makes memory executable. The feature matters because the monitor only records the process tree it launched. Once the work moves elsewhere, the trace goes quiet even though the sample is still encrypting. That silence would otherwise look like inactivity. `w_api_rate` supplies a reference for overall intensity, and `c_duration_s` gives the cumulative counters a denominator.

Features are computed over the whole process tree rooted at the sample, because the benign workloads spawn their utilities as child processes. Path arguments are classified by storage location. The classifier resolves both the mapped drive letter and the UNC forms that appear in the trace once the path has been resolved. Table IV lists the 18 features in the order used above.

TABLE IV
HOST FEATURES DERIVED FROM THE API CALL TRACE

Feature	Description
<i>1) Locating targets</i>	
<code>w_traversal_rate</code>	<code>FindFirstFile*</code> , <code>FindNextFile*</code> and <code>NtQueryDirectoryFile*</code> , per second
<code>c_traversal</code>	Directory enumeration calls since launch
<i>2) Clearing obstacles</i>	

Feature	Description
c_handle_enum	NtQuerySystemInformation, CreateToolhelp32Snapshot and Process32First/NextW since launch
w_reg_rate	RegSetValueEx*, RegCreateKeyEx*, NtSetValueKey and NtCreateKey, per second
c_reg_write	Registry writes since launch
3) Generating keys	
c_keygen	CryptGenKey, CryptAcquireContext*, BCryptGenerateSymmetricKey and related, since launch
4) The encryption loop	
w_open_rate	NtOpenFile and NtCreateFile carrying a path, per second
w_write_rate	NtWriteFile, WriteFile and NtCreateFile carrying a corpus path, per second
w_rbo_rate	Distinct files read and later written, renamed or deleted, per second
w_write_buf_entropy	Mean Shannon entropy of the Buffer argument of each write landing on a corpus file
w_iobuf_entropy_delta	Mean write-buffer entropy minus mean read-buffer entropy in the same window
w_mmap_rate	NtCreateSection, NtMapViewOfSection and NtUnmapViewOfSection, per second
5) Disposing of the original	
w_rename_rate	MoveFileWithProgress* and NtSetInformationFile where the target name is the source name with an element appended, per second
w_delete_rate	DeleteFileW, NtDeleteFile and NtSetInformationFile flagging a file for deletion, per second
6) Organisation and normalisation	
c_thread	NtCreateThreadEx, CreateThread and CreateRemoteThread* since launch
c_injection	1 if WriteProcessMemory, VirtualAllocEx or an RWX protection change was observed
w_api_rate	Every call recorded in the window, per second
c_duration_s	Seconds elapsed since the first recorded call

2) Network Features:

The same sequence reaches the network in a different form. A probe sees SMB2 messages, their command codes, and the bytes each one carries.

1) Locating targets. A client walking the tree appears on the wire as directory queries.

w_smb_querydir_rate and c_smb_querydir count them.

2) Opening targets. Each file the sample touches must first be opened, and `w_smb_create_rate` and `c_smb_create` count those opens. Alongside these, `w_smb_shortcode_rate` and `c_smb_shortcode` count every request carrying no data payload. Opening, closing and querying a file transfer no content at all, so these counters recover what byte volume hides. A client moving one large file and a client processing several small ones can transfer identical volume while differing by orders of magnitude in file count.

3) The encryption loop. Berrueta et al. reduce this stage to byte volume per direction [3][17]. Reads and writes are kept apart, because their relationship is what identifies encryption. A client that pulls a file down and pushes equivalent amount back is transforming it. A client that only pulls is reading. A client that only pushes is uploading. A single combined byte count would collapse all three into one number. These quantities appear as `w_smb_read_bps` and `w_smb_write_bps`, with `c_smb_read_bytes` and `c_smb_write_bytes` as their cumulative forms.

4) Disposing of the original. Disposal is counted two ways, for the same reason as on the host. The four patterns dispose of the original in one of two ways: those that overwrite it in place finish with a rename, and those that write out a new file finish by deleting it [16]. The wire distinguishes the two by the information level inside `SET_INFO`, so `w_smb_rename_rate` and `w_smb_delete_rate` count each separately. They are also summed into `w_smb_destructive_rate` and accumulated into `c_smb_destructive`.

5) Packet-length identifiers. Every feature above requires the SMB2 command field, which encryption hides. The features in this group do not. They identify an operation by frame length alone. This works when a message carries no variable-length field, because its size is then fixed by the specification. Hatungimana et al. describe seven SMB frame lengths recognisable[18]. Six of them they designate Commands of Interest. The seventh, the 260-byte packet, they use instead

to delimit what they call a Region of Interest. The `coi_` prefix is kept here for all seven, five of which are adopted as features.

6) Byte volume and direction. This group reads only the frame header, which encryption also leaves exposed. Volume matters because encryption is bulk movement. A family working through 800 files transfers the share twice, once in each direction. Direction separates that movement from every other reason a client moves data. `w_smb_byte_ratio` divides outbound bytes by inbound bytes. A client that reads a file and writes back an encrypted version of the same size returns roughly what it received, so the ratio sits near one. A client reading the share to hash or index it sends almost nothing back, so the ratio sits below one. A client uploading new content sits above one. Ransomware is the case that sits at one while moving a large volume.

Cumulative counters are anchored at the client's most recent TCP connection to port 445. An off-path probe operates continuously, and a connection opening is the only origin it observes. Table V lists the twenty-four features in the order used above.

TABLE V
NETWORK FEATURES DERIVED FROM THE PACKET CAPTURE

Feature	Description
1) Locating targets	
<code>w_smb_querydir_rate</code>	QUERY_DIRECTORY requests, smb2.cmd 14, per second
<code>c_smb_querydir</code>	QUERY_DIRECTORY requests, smb2.cmd 14, since the connection opened
2) Opening targets	
<code>w_smb_create_rate</code>	CREATE requests, smb2.cmd 5, per second
<code>c_smb_create</code>	CREATE requests, smb2.cmd 5, since the connection opened
<code>w_smb_shortcmd_rate</code>	Requests carrying no data payload, smb2.cmd 5, 6, 14, 16 and 17, per second
<code>c_smb_shortcmd</code>	Requests carrying no data payload, same command codes, since the connection opened
3) The encryption loop	
<code>w_smb_read_bps</code>	smb2.read_length summed over READ responses, smb2.cmd 8, per second
<code>w_smb_write_bps</code>	smb2.write_length summed over WRITE requests, smb2.cmd 9, per second

Feature	Description
c_smb_read_bytes	smb2.read_length, smb2.cmd 8, since the connection opened
c_smb_write_bytes	smb2.write_length, smb2.cmd 9, since the connection opened
4) Disposing of the original	
w_smb_rename_rate	SET_INFO requests, smb2.cmd 17, with information level 10, per second
w_smb_delete_rate	SET_INFO and CLOSE, smb2.cmd 17 and 6, with information level 13, per second
w_smb_destructive_rate	The two preceding quantities summed
c_smb_destructive	Renames and deletions, information levels 10 and 13, since the connection opened
5) Packet-length identifiers	
w_coi_querydir_rate	Frames of exactly 260 bytes, per second - a QUERY_DIRECTORY request
w_coi_read_rate	Frames of exactly 171 bytes, per second - a READ request
w_coi_write_rate	Frames of exactly 138 bytes, per second - a WRITE response
w_coi_close_rate	Frames of exactly 146 bytes, per second - a CLOSE request
w_coi_setinfo_rate	Frames of exactly 124 bytes, per second - a SET_INFO response
6) Byte volume and direction, requiring no SMB field	
w_smb_bytes_out_bps	All frame bytes sent to the server, per second
w_smb_bytes_in_bps	All frame bytes received from the server, per second
w_smb_byte_ratio	Outbound bytes divided by inbound, each offset by one
c_smb_bytes_out	Outbound frame bytes since the connection opened
c_smb_bytes_in	Inbound frame bytes since the connection opened

D. Window Formulation

A deployed detector does not know when an execution will end. It cannot compute any quantity over a complete run. It must instead decide repeatedly on the evidence available at each point in time. Every window of a ransomware execution is labelled malicious, and every window of a benign execution is labelled benign.

The window is ten seconds wide and advances in one-second steps. One window therefore ends at every second. Fig. 2 shows the sequence.

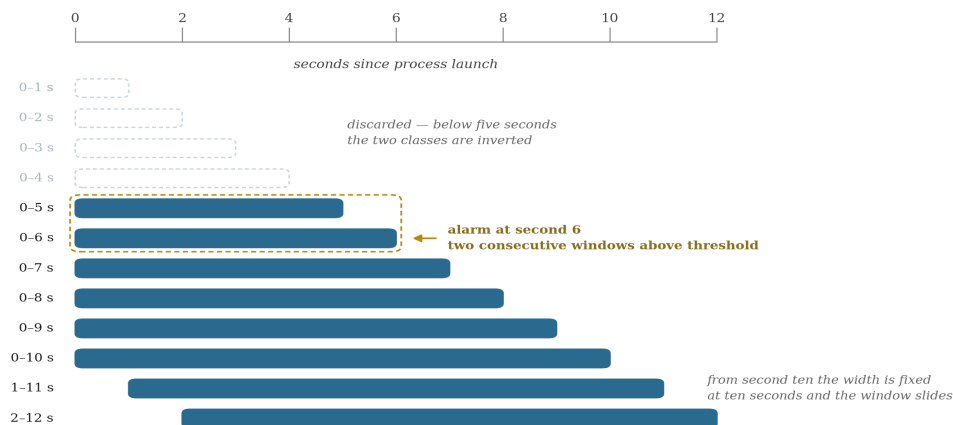


Fig. 2. Window formulation.

A window of fixed width cannot be scored until it has filled. The first decision would therefore not be available until second ten. Two consecutive windows are required for an alarm, so the earliest possible alarm would fall at second eleven. That is too late for the families that begin encrypting at once. Dharma, for example, destroys its hundredth file at second 3.4. The first ten seconds are therefore divided into shorter windows, so that a decision can be made while the damage is still small. From second ten the width is fixed and the window slides.

The first window scored is the one closing at second five. Those closing at seconds one through four are computed and then discarded. A ransomware execution does not begin encrypting when it starts. It first enumerates the file system, establishes persistence, contacts external servers and disables what protection it can. Only then does it enter the encryption loop [26].

A window closing before second five is therefore observing preparation and not encryption. The measurements reported here agree with that. At two seconds ransomware writes at a median of 8.5 files per second against 14.0 for the benign workloads. The ordering only reverses at five seconds, at 40.4 against 20.0.

E. Model Training and Evaluation

Evaluation follows a leave-one-family-out scheme. For each of the ten families, a model is trained on the executions of the remaining nine. It is then tested on the family held out of training. The procedure is repeated ten times, and the reported figures aggregate the ten test folds.

The reason for this scheme is that ransomware families are highly derivative. Two samples of the same family may differ by little more than a packer. Random cross-validation would then place nearly duplicated samples on both sides of the partition. The resulting figure would not describe performance on a new family. Pendlebury et al. show that similarities among samples of the same family make k-fold cross-validation positively biased, because the training set contain at least one member of every family while a real deployment meets new ones [27]. Arp et al. find it among the most common problems in security machine learning research [28]. Holding out a whole family is the corrective. It does not address temporal drift, which degrades a deployed model as families evolve after training [29], [30]. The training and evaluation procedure is as follows.

- **Classifier:** LightGBM, a gradient-boosted tree ensemble. It is applied to three feature subsets: host features alone, network features alone, and the two combined.
- **Partitioning:** Ten leave-one-family-out folds. Benign executions are assigned round-robin within each workload type, are never divided across the train and test partitions, and the decision threshold is selected on the training fold only.
- **Decision rule:** An execution is counted as detected when a specified number of consecutive windows exceed the threshold. Section IV-A reports the value selected from it.

- **Reporting:** Results are reported per execution rather than per window. Adjacent windows share most of their content and a model that is correct on one window is therefore correct on nearly all of them. A per-window accuracy would reflect that redundancy more than the ability to identify an execution. Detection rate and false-positive rate are aggregated across the ten test folds.

Reassigning the labels at random at the level of the execution collapses the model to near-random behaviour. No structure in the fold assignment or in the feature construction therefore allows the true labels to be recovered.

F. Limitations of the Design

1) The Benign Corpus:

The benign corpus is the principal constraint on every false-positive figure reported in this work. It is also weaker than the other benign corpus used in the comparable literature. ShieldFS collected one month of real filesystem activity from eleven machines [4]. Berrueta et al. collected 2,527 hours of real user traffic on a campus LAN [17]. Of that total, 2,477 hours form their false-positive validation set. Neither resource was available for this project, so the workloads described in Section III-B were built in their place. The point is general rather than particular to this work as Sommer and Paxson argue that in security machine learning the benign side of an evaluation binds hardest, because it fixes what a false alarm means [31].

Two gaps follow from that substitution. First, the corpus contains no workload of extended duration. A scheduled backup or a full antivirus scan sustains heavy I/O across a share for minutes or hours. No workload in this corpus runs for more than a few minutes. Second, no interactive application is represented. The corpus contains no word processor writing to the

share, no search indexer, and no synchronisation client. These are the workload classes that most closely resemble ransomware at the I/O level.

2) Instrumentation and Environment:

The endpoint channel is instrumented with a sandbox monitor. The monitor hooks user-mode calls within the process tree it launches. It provides no kernel-level filtering and no visibility outside that tree. It also stops recording a given API once a fixed number of calls to it have been logged. A deployed agent with kernel visibility would observe more than this monitor does.

The file server is Samba. Samba underpins most commercial network-attached storage appliances and is standard in mixed environments. Windows Server nevertheless remains more common in large enterprises, and Section IV-D shows that two of the published packet-length signatures do not transfer between the two implementations.

The file server is also not a separate machine. Samba runs on the analysis host itself, and the guest reaches it across a virtual bridge instead of a switched network. Round-trip latency is therefore far below what any production path imposes. A ransomware write rate here would be slower on a real segment, so the loss figures in Section IV-C are an upper bound.

The share also carries no traffic except the sample's. A departmental server carries the activity of many clients at once. Under that condition the single false positive of Section IV-A, a workload running several utilities in parallel, would become more common.

Host features also count operations on both storage locations, because 11 of the 18 count calls that carry no path argument. This design matches deployment conditions. A workstation that mounts a share also runs Windows and holds a user profile, and ransomware that reaches that

workstation encrypts both. A machine holding no local data of value still falls outside what this corpus evaluates.

The testbed contains a single client. It therefore cannot evaluate the case in which no agent can be installed at all. In an incident investigated by S-RM and reported in March 2025, Akira operators had their payload quarantined by an EDR tool on a Windows server. They then scanned the internal network for alternatives and identified an unsecured webcam running a lightweight Linux operating system compatible with their Linux variant. They ran the Linux encryptor on the webcam itself, mounted the Windows shares from it, and encrypted those shares over SMB. The activity went unobserved because the device carried no security monitoring [32].

IV. EXPERIMENTAL RESULTS

This section presents the experimental results. Every reported value comes from a single configuration. The corpus holds 109 ransomware executions across ten families and 187 benign executions. Each observation window carries 42 features. Windows are 10 seconds wide, with a five-second minimum. An alarm is raised when two consecutive windows exceed the threshold, which places the decision 6 seconds after process launch. Section IV-A compares the two observation channels. Section IV-B reports the feature importance. Section IV-C reports detection latency and the data loss that accompanies it. Section IV-D reports the effect of protocol encryption.

A. RQ1: Contribution of the Network Channel

Table VI reports detection and false-positive rates for the three models. The rows vary the number of consecutive windows required before an alarm is raised. Each cell gives the detection rate first and the false-positive rate second.

TABLE VI
DETECTION AND FALSE-POSITIVE RATE AGAINST THE CONFIRMATION REQUIREMENT

k	Latency	Host-only	Network-only	Combined
1	5 s	100 % / 2.1 %	100 % / 6.4 %	100 % / 0.5 %
2	6 s	100 % / 2.1 %	100 % / 2.1 %	100 % / 0.5 %
3	7 s	100 % / 1.6 %	100 % / 1.6 %	100 % / 0.5 %
5	9 s	100 % / 1.1 %	100 % / 1.6 %	100 % / 0.5 %

Every model detects every execution at every threshold evaluated. The three models differ only in how many benign workloads they misclassify as ransomware. With 187 benign runs, one false alarm is worth about 0.5%. The whole spread across the three models is therefore a small number of samples. Raising the number of consecutive windows lowers the rate for both single-channel

models, which is the expected behaviour. The benign workloads produce bursts and then stop, whereas ransomware sustains its intensity for as long as targets remain. From two windows onward the two single-channel models are equivalent, and the combined model stays at one false alarm throughout.

The results that follow require two consecutive windows. A single window is enough for detection, but it leaves the network channel at 6.4%, because one noisy interval on an active benign workload is enough to trigger it. Three windows cuts both single-channel rates, and five cuts the host rate further, at a cost of one and three seconds of extra delay. Neither improves the combined model, which already sits at one false alarm. Two windows is the earliest point at which the confirmation requirement is effective.

The answer to RQ1 is that combining the channels reduces false alarms without affecting detection. The result is limited by the size of the corpus. With 187 benign executions a single false alarm is worth half a percentage point, so the entire spread between the best and the worst configuration is three samples. A larger benign corpus, drawn from production activity rather than scripted workloads, would settle the comparison at a granularity this one cannot.

Two points hold regardless of corpus size. First, every configuration detected every execution, so the choice between channels is not a choice about whether ransomware is found. Second, the significance lies in what each channel can observe. The endpoint sees local encryption that never traverses the wire. The wire continues to observe a sample that has somehow succeeded in evading the endpoint agent.

B. Feature Importance

Fig. 3 reports the importance each feature of the combined model contributes in the window in which the alarm is raised, averaged over the ten folds. Table VII sets the same quantity beside the two single-channel models.

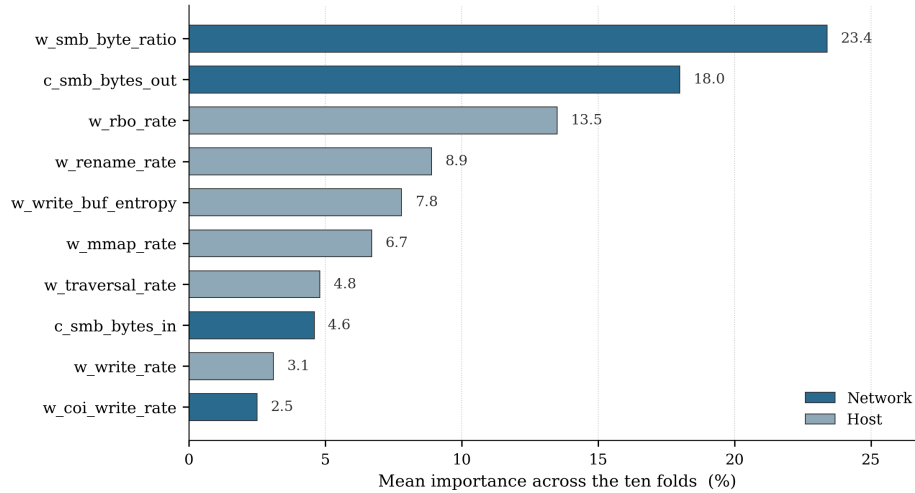


Fig. 3. Feature importance of the combined model in the window in which the alarm is raised, averaged over the ten leave-one-family-out folds.

TABLE VII
FEATURE IMPORTANCE IN THE WINDOW IN WHICH THE ALARM IS RAISED, AVERAGED OVER TEN FOLDS

#	Combined model	Importance	Host-only	Importance	Network-only	Importance
1	w_smb_byte_ratio	23.4 %	w_traversal_rate	26.4 %	w_smb_byte_ratio	24.0 %
2	c_smb_bytes_out	18.0 %	w_mmap_rate	18.1 %	c_smb_bytes_out	23.2 %
3	w_rbo_rate	13.5 %	w_rename_rate	13.9 %	w_coi_write_rate	14.0 %
4	w_rename_rate	8.9 %	w_write_buf_entropy	10.7 %	w_smb_rename_rate	6.7 %
5	w_write_buf_entropy	7.8 %	w_write_rate	9.9 %	c_smb_bytes_in	5.7 %
6	w_mmap_rate	6.7 %	w_rbo_rate	7.0 %	w_smb_bytes_in_bps	5.0 %

Feature importance is measured as the reduction in training loss attributable to each feature across the ensemble, averaged over the ten folds. It is reported for the window in which the alarm is raised. At the moment of decision the balance is 52.6% network against 47.4% host.

The two single-channel models reach the same false-positive rate at two windows, and they reach it on different evidence. No feature appears in both lists. The endpoint channel observes

how the samples work. Its features cover the rate at which the sample traverses the tree and whether it maps files instead of writing them. They also cover whether the names it produces are the original names with an element appended, and the character of the bytes it writes. The network channel observes the shape of the traffic. Its features cover the ratio of transmitted to received volume, the total volume transferred, and the number of operations of each length that have crossed the wire.

This is the concrete form of the answer to RQ1. No feature ranks among the most important for both channels, so the two are measuring different things. What they share is sufficiency because each alone finds every execution, and combining them reduces the false alarms.

C. RQ2: Detection Latency and Data Loss

The alarm is raised 6 seconds after process launch. The cost of that delay depends on the family. Loss is measured from the pcap capture. Each file is charged once, at its full original size, at the point the server is first asked to access it. This follows the convention of Berrueta et al., which charges the whole file even when only part of it was encrypted [17]. Only the share is counted, because the capture cannot see local storage.

Table VIII reports the results by family.

TABLE VIII
FILES AND MEGABYTES DESTROYED ON THE SHARE AT THE POINT THE ALARM IS RAISED

Family	n	Files (median)	Files (p90)	MB (median)	MB (max)	Lost nothing
7ev3n	7	0	0	0.0	0.0	7/7
Conti	8	0	0	0.0	0.0	8/8
REvil	13	0	0	0.0	0.0	13/13
MedusaLocker	14	0	0	0.0	27.5	13/14
Maze	10	0	39	0.0	17.4	8/10
AvosLocker	7	0	21	0.0	22.6	5/7

Family	n	Files (median)	Files (p90)	MB (median)	MB (max)	Lost nothing
Akira	11	0	63	0.0	28.6	6/11
Dharma	14	144	182	82.0	107.4	0/14
RagnarLocker	13	144	144	82.0	82.0	1/13
Nefilim	12	415	415	254.5	254.5	1/12

The distribution is bimodal and has no meaningful central value. In 62 of 109 executions the sample has destroyed nothing when the alarm is raised. Among the 47 executions that do lose data, the mean is 168 files and 97.4 MB. Across the complete set the mean is 77 files and 44.8 MB. The median is zero, and the ninetieth percentile is 185 files and 106.8 MB.

The difference is from where each family spends its opening seconds. Three families are already encrypting the share when the alarm is raised. Nefilim writes to it at a median of 14.8 MB per second, RagnarLocker at 8.0 and Dharma at 2.4. By that point they have transferred 78, 42 and 15 MB respectively.

All three also rename at scale. They rename at 58, 28 and 126 files per second against a benign median of zero, and their read-before-overwrite rates fall between 28 and 52. These three families are the ones that cost data. Only 2 of their 39 executions reach the alarm having destroyed nothing.

Two further families work at comparable intensity but reach the share inconsistently.

AvosLocker writes at a median of 575 files per second across both storage locations, and Akira at 300. The median SMB write rate, however, is 375 KB per second for AvosLocker and zero for Akira. Only five of eleven Akira executions have written to the share at all by second six.

Both families also split within themselves, and for different reasons. Every Akira execution touches local storage within milliseconds of launch. The point at which it first reaches the mapped drive is far more variable, ranging from 4.2 to 18.9 seconds across the eleven samples.

Five cross before the alarm and six do not, and those six are exactly the executions that lose nothing.

AvosLocker behaves differently. Most of its executions touch both drives at once, within milliseconds, and five of seven are writing to the share by second six. Only two of the seven have encrypted a corpus file. The rest are still enumerating the tree, which accounts for the several megabytes they have sent to the server without destroying anything.

The remaining five families have written almost nothing to either location at second six.

MedusaLocker maps memory sections at a median of 236 per second against a benign median of 0.6. It transforms each file in place with CPU instructions, so the encryption never passes through the write API. 13 of its 14 executions reach the alarm without loss.

Maze maps at a lower rate and writes at 0.2 files per second, and eight of ten lose nothing. Conti and 7ev3n write at 2.2 and 9.0 files per second with every share counter at zero, and every one of their executions reaches the alarm intact. REvil is the same. Its opening seconds are dominated by registry activity at a median of 3.0 operations per second, against a benign median of zero. During those seconds it reads the configuration that determines what it will encrypt.

One family in the second group is notable because the content it writes differs from what reaches disk. Akira encrypts only the leading portion of each file. Across 388 files larger than 50 KB, compared before and after the same execution, the entropy of the first block rises from 6.44 to 7.95 bits per byte. The tail moves only from 4.92 to 5.52. Ineza et al. show that a content-based detector trained only on unencrypted and fully encrypted files degrades to roughly chance at ten percent coverage [33]. The technique does not defeat the measurement used here, which reads the buffer supplied to each write call. That buffer registers 7.17 bits for Akira, against 7.19 for Maze, which encrypts uniformly.

The second part of RQ2 concerns the trade-off between the confirmation requirement and the loss it permits. Table IX reports both quantities across the confirmation thresholds evaluated in Section IV-A.

TABLE IX
DATA LOSS AGAINST THE CONFIRMATION REQUIREMENT, WITH THE CORRESPONDING FALSE-POSITIVE RATE OF THE FUSED MODEL

k	Latency	Files (mean)	Files (p90)	MB (mean)	Lost nothing	False positives
1	5 s	69.3	144	40.4	69/109	0.5 %
2	6 s	77.2	185	44.8	62/109	0.5 %
3	7 s	95.9	342	57.4	59/109	0.5 %
5	9 s	140.6	517	94.7	58/109	0.5 %

Raising the requirement from one window to five delays the decision by four seconds. The mean loss rises from 69 files to 141, and the ninetieth percentile rises from 144 to 517, which is more than half the contents of the share. The false-positive rate of the combined model is 0.5% at every point in that range.

The trade-off on this corpus is therefore one-directional. Additional confirmation costs data and returns nothing. Families that spend their opening seconds preparing lose nothing at any threshold, while those that begin writing immediately lose more at each step.

D. RQ3: The Encrypted Protocol

RQ3 asks which network features remain computable when the file-sharing protocol is encrypted, and what the loss of the remainder costs. Berrueta et al. build a detector for exactly that setting, using only packet arrival times, sizes and direction [17]. What has not been measured is which SMB frame lengths survive encryption, by how much they shift, and how closely their frequencies hold. This section answers that in three steps. It first tests whether the packet-length signatures reproduce against other server implementation. It then measures how

those lengths behave under SMB3 encryption. It finally reports what the lost features cost in false alarms.

1) Portability Across Server Implementations:

Hatungimana et al. describe seven SMB frame lengths that a probe can recognise [18]. All seven are evaluated here.

Their measurements were obtained against a Windows Server 2016 file server with a Windows 10 client and a share of 200 files across eight directories. The measurements here were obtained against Samba, with 800 files in a tree six to seven levels deep. They cover all 109 executions. The published identifiers correspond to unencrypted traffic. Table X reports what actually carries each published length.

TABLE X
PUBLISHED PACKET-LENGTH SIGNATURES MEASURED ACROSS ALL 109 EXECUTIONS

Signature	Published [18]	Frames at that length	Command at that length
Read request	171 B	216,432	100.00 % are READ requests
Write response	138 B	2,188,390	99.99 % are WRITE responses
Close request	146 B	5,104,656	100.00 % are CLOSE requests
Set-info (rename)	124 B	707,430	100.00 % are SET_INFO responses
Directory enumeration	260 B	2,176,994	99.97 % are two compounded QUERY_DIRECTORY requests
Create response	410 B	95,609	78.8 % CREATE + QUERY_INFO, 11.4 % CREATE + SET_INFO, 7.8 % CREATE + CLOSE
Open response	378 B	219,941	95.5 % CREATE + QUERY_INFO, 2.3 % a lone CREATE response

The difference between the signatures that reproduce and those that do not follows from the protocol specification. An SMB2 message carries a 64-byte header of its own and four layers sit below it on the path used here: Ethernet, IPv4, TCP, and the four-byte Direct TCP transport header that precedes SMB on port 445. That transport header occupies the same position as the

NetBIOS session header. Together they add 122 bytes, assuming a TCP header that carries no options. Fig. 4 shows the composition. A signature is fixed when the message carries no variable-length content and that property makes it possible to predict which lengths will transfer. The header length, the StructureSize values and the status code cited here are all fixed by the protocol specification [34].

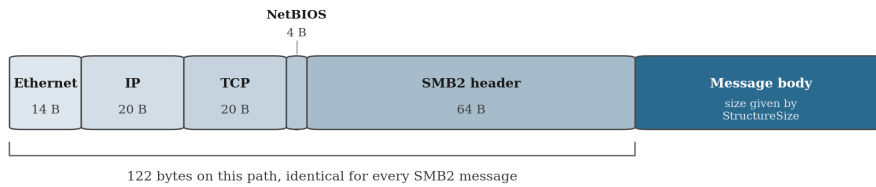


Fig. 4. Composition of an SMB2 frame carrying no variable-length content.

TABLE XI
STRUCTURE SIZE CARRIED IN EACH MESSAGE, AGAINST THE FRAME LENGTH MEASURED

Message	Body length (bytes)	122 + Body length	Measured
SET_INFO response	2	124	124 B
WRITE response	16	138	138 B
CLOSE request	24	146	146 B
READ request	49	171	171 B

4 of the 7 signatures hold in both directions. A frame of 171 bytes is a read request in every one of the 216,432 cases observed. The close request is exact in both directions across 5.1 million frames. The write response is correct in 99.99% . The set-info response is exact at 124 bytes.

One further correction is required to the published set. Hatungimana et al. give 124 bytes as a client set-information request. Decoding here shows that it is the server's SET_INFO response.

A SET_INFO request carries the new filename in a variable-length buffer and therefore has no fixed size at all, whereas the SET_INFO response body is the 2-byte StructureSize field alone.

The directory-enumeration signature holds for a different reason, which the original description does not give. Hatungimana et al. present 260 bytes as the length of a single packet [18].

Decoding shows that 99.97% of the 2,176,994 frames at that length carry two

QUERY_DIRECTORY requests compounded into one frame, with the command and structure-size fields each holding two values. The pair arises from how directory enumeration works. A client must issue a second QUERY_DIRECTORY request to learn that the directory has been exhausted, because the server signals this by returning STATUS_NO_MORE_FILES. Windows then compounds the two into a single frame.

The two create-response signatures do not hold at all. A create response carries optional create contexts, so its body has no fixed size and the server decides what the frame will measure.

2) Behaviour Under Encryption:

To measure the effect of encryption directly, a Dharma execution and a Maze execution were repeated against a share requiring SMB3 encryption. Command identification is eliminated. Encryption therefore removes 14 of the 24 network features, since each of them is computed from the command field. Table XII reports the behaviour of the 5 surviving signatures. Each row gives a signature, its plaintext and encrypted lengths, and the frame counts before and after for the same sample.

TABLE XII
SURVIVING PACKET-LENGTH SIGNATURES UNDER SMB3 ENCRYPTION

Signature	Plaintext	Encrypted	Dharma frames	Maze frames
Read request	171 B	223 B	997 → 1,001	3,441 → 3,489
Write response	138 B	190 B	1,815 → 1,830	15,068 → 15,069
Close request	146 B	198 B	37,733 → 37,951	59,123 → 59,144
Directory enumeration	260 B	312 B	27,985 → 27,623	13,975 → 13,977
Set-info	124 B	176 B	3,043 → 3,074	832 → 832

All five signatures shift by exactly 52 bytes. That is the size of the transform header SMB3 applies to each message. The frequencies are preserved as well, and the largest discrepancy is

1.4%. Byte volume is likewise nearly unchanged. The Dharma execution transferred 285.6 MB outbound and 229.5 MB inbound on the plaintext protocol, against 290.6 MB and 235.4 MB under encryption. The Maze execution transferred 747.5 MB and 623.4 MB against 755.5 MB and 631.6 MB.

The five surviving lengths correspond to features already present in the feature set:

`w_coi_read_rate`, `w_coi_write_rate`, `w_coi_close_rate`, `w_coi_querydir_rate` and `w_coi_setinfo_rate`. Under an encrypted protocol each one is computed at the shifted length. The five byte-volume features require no adjustment at all, because frame size and direction are unchanged. These are `w_smb_bytes_out_bps`, `w_smb_bytes_in_bps`, `w_smb_byte_ratio`, `c_smb_bytes_out` and `c_smb_bytes_in`.

3) Cost of the Loss:

Table XIII reports network-only performance under the two observability conditions. The first is the full 24-feature set on a plaintext protocol. The second is the 10-feature set that survives encryption, made up of the 5 packet-length signatures and the 5 byte-volume features.

TABLE XIII
NETWORK-ONLY PERFORMANCE UNDER EACH OBSERVABILITY CONDITION

Observability condition	Features	Detection	False positives
Plaintext SMB2, full feature set	24	100.0 %	2.1 %
Encrypted: lengths and byte volume	10	100.0 %	4.3 %

The loss of 14 of the 24 features costs no detection. The encrypted feature set still detects every execution but the false-positive rate rises from 2.1% to 4.3%. The 10 surviving features remain informative because ransomware is anomalous on two axes at the same time. It issues a large number of operations, and each operation transfers a complete file. The packet-length signatures

count the operations, and the byte-volume features measure the movement, and neither axis depends on the command field.

The answer to RQ3 is therefore twofold. 10 of the 24 network features survive an encrypted protocol. Detection on this corpus is unaffected, and the cost of the loss is a false-positive rate about twice as high.

V. CONCLUSION

Host-based and network-based ransomware detection have developed as separate lines of work, each evaluated on its own corpus. This thesis built a laboratory in which the same executions produce both records, so the two channels could be compared under an identical corpus, window formulation and evaluation protocol.

The answer to RQ1 is that adding the network channel helps reduce false alarm. Every configuration detects every execution at every confirmation threshold tested. Neither channel is redundant, because the evidence they rely on does not overlap. The endpoint watches how a sample works, and the wire watches the shape of the traffic. The choice between them is therefore not about accuracy but about reach.

RQ2 shows that six seconds falls inside the preparation phase of most families. In 62 of 109 executions nothing on the share has been destroyed when the alarm is raised. The distribution is bimodal. 7 of the 10 families spend their opening seconds enumerating, mapping files into memory or reading configuration, while three begin writing within the first second. Waiting longer does not help because extending the confirmation requirement from one window to five doubles the mean loss and leaves the false-positive rate unchanged.

RQ3 shows that protocol encryption is survivable on the network channel. 4 of the 7 packet-length signatures hold in both directions against the other server implementation. A fifth holds for a different reason, since the Windows client compounds its directory requests in pairs. Under encryption the surviving lengths shift by exactly 52 bytes and their frequencies hold to within

1.4%. A detector built from the ten remaining features finds every execution at 4.3% false-positive rate against 2.1% on plaintext.

Four limitations bound these conclusions. The corpus is small and with one false alarm worth half a percentage point, the separations reported between configurations are therefore indicative rather than precise. The benign corpus also contains nothing that sustains heavy I/O for hours. The file server runs on the analysis host, so the loss figures are an upper bound. And with a single client the testbed cannot evaluate the case in which no agent can be installed at all.

VI. FUTURE WORK

The benign corpus is the binding constraint and the most valuable thing to replace. Two additions are needed. The first is a workload that sustains heavy I/O over a long period, such as a scheduled backup or a full antivirus scan. None of these is represented here, and the confirmation rule would need to be re-examined against them. The second is an interactive application writing to the share, since a word processor saving a document produces content with higher entropy than what the benign samples in this corpus writes.

A multi-client testbed would answer the question this one cannot. The comparison in Section IV-A is between an agent and a probe on the same machine, and it establishes only that the probe adds little where the agent runs. It says nothing about the case where the agent cannot run. A testbed with several clients, at least one carrying no agent, would measure what the probe recovers there.

A kernel-level monitor would remove a known gap. The sandbox monitor used here hooks user-mode calls within a single process tree and stops recording an API after a fixed number of calls. An ETW consumer observes what this does not, and would establish whether the endpoint figures understate what a deployed agent achieves.

Finally, the corpus contains no adversarial execution. No sample was observed throttling its I/O below the intensity the model keys on, or issuing system calls directly to bypass the monitor. The modern families excluded here for requiring command-line parameters are the ones most likely to carry such techniques. Obtaining the configuration needed to run them would extend the corpus in the direction most likely to challenge the detector rather than confirm it. Chen et al.

pose the harder version of this question, which is whether a detector still works against samples modified specifically to defeat it [35].

REFERENCES

- [1] Verizon. "2025 Data Breach Investigations Report." Verizon Business, 2025.
- [2] Arányi, G., Miseta, T., and Szücs, V. "Ransomware detection based on server-side file operation logs using machine learning." *Journal on Information Security*, vol. 2026, 2026, article 8.
- [3] Morató, D., Berrueta, E., Magaña, E., and Izal, M. "Ransomware early detection by the analysis of file sharing traffic." *Journal of Network and Computer Applications*, vol. 124, 2018, pp. 14–32.
- [4] Continella, A., Guagnelli, A., Zingaro, G., De Pasquale, G., Barengi, A., Zanero, S., and Maggi, F. "ShieldFS: A Self-healing, Ransomware-aware Filesystem." *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, 2016, pp. 336–347.
- [5] Scaife, N., Carter, H., Traynor, P., and Butler, K. R. B. "CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data." *Proceedings of the 36th IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2016, pp. 303–312.
- [6] Kharraz, A., and Kirda, E. "Redemption: Real-Time Protection Against Ransomware at End-Hosts." *Research in Attacks, Intrusions, and Defenses (RAID 2017)*, *Lecture Notes in Computer Science*, vol. 10453, Springer, 2017, pp. 98–119.
- [7] Mehnaz, S., Mudgerikar, A., and Bertino, E. "RWGuard: A Real-Time Detection System Against Cryptographic Ransomware." *Proceedings of the 21st International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, 2018, pp. 114–136.
- [8] Kolodenker, E., Koch, W., Stringhini, G., and Egele, M. "PayBreak: Defense Against Cryptographic Ransomware." *Proceedings of the 2017 ACM Asia Conference on Computer and Communications Security (ASIACCS)*, 2017, pp. 599–611.
- [9] Begovic, K., Al-Ali, A., and Malluhi, Q. "Cryptographic ransomware encryption detection: Survey." *Computers & Security*, vol. 132, 2023, 103349.
- [10] Berrueta, E., Morató, D., Magaña, E., and Izal, M. "A Survey on Detection Techniques for Cryptographic Ransomware." *IEEE Access*, vol. 7, 2019, pp. 144925–144944.
- [11] Vehabovic, A., Ghani, N., Bou-Harb, E., Crichigno, J., and Yayimli, A. "Ransomware Detection and Classification Strategies." *Proceedings of the 2022 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, 2022.
- [12] Kok, S. H., Abdullah, A., Jhanjhi, N. Z., and Supramaniam, M. "Prevention of Crypto-Ransomware Using a Pre-Encryption Detection Algorithm." *Computers*, vol. 8, no. 4, 2019, article 79.
- [13] Alqahtani, A., and Sheldon, F. T. "Temporal Data Correlation Providing Enhanced Dynamic Crypto-Ransomware Pre-Encryption Boundary Delineation." *Sensors*, vol. 23, no. 9, 2023, article 4355.

- [14] McIntosh, T., Jang-Jaccard, J., Watters, P., and Susnjak, T. "The Inadequacy of Entropy-Based Ransomware Detection." *Neural Information Processing (ICONIP 2019), Communications in Computer and Information Science*, vol. 1143, Springer, 2019, pp. 181–189.
- [15] Davies, S. R., Macfarlane, R., and Buchanan, W. J. "Differential area analysis for ransomware attack detection within mixed file datasets." *Computers & Security*, vol. 108, 2021, 102377.
- [16] Ahmed, M. E., Kim, H., Camtepe, S., and Nepal, S. "Peeler: Profiling Kernel-Level Events to Detect Ransomware." *Computer Security – ESORICS 2021, Lecture Notes in Computer Science*, vol. 12972, Springer, 2021, pp. 240–260.
- [17] Berrueta, E., Morató, D., Magaña, E., and Izal, M. "Crypto-ransomware detection using machine learning models in file-sharing network scenarios with encrypted traffic." *Expert Systems with Applications*, vol. 209, 2022, 118299.
- [18] Hatungimana, G., Mahmood, A. N., and Chowdhury, M. J. M. "A Hybrid Framework for Crypto-Ransomware Detection in Enterprise Shared Storage." *arXiv preprint arXiv:2606.30586*, 2026.
- [19] Papadogiannaki, E., and Ioannidis, S. "A Survey on Encrypted Network Traffic Analysis Applications, Techniques, and Countermeasures." *ACM Computing Surveys*, vol. 54, no. 6, 2021, article 123.
- [20] Wang, Z., Fok, K.-W., and Thing, V. L. L. "Machine learning for encrypted malicious traffic detection: Approaches, datasets and comparative study." *Computers & Security*, vol. 113, 2022, 102542.
- [21] Berrueta, E., Morató, D., Magaña, E., and Izal, M. "Open Repository for the Evaluation of Ransomware Detection Tools." *IEEE Access*, vol. 8, 2020, pp. 65658–65669.
- [22] Davies, S. R., Macfarlane, R., and Buchanan, W. J. "NapierOne: A modern mixed file data set alternative to Govdocs1." *Forensic Science International: Digital Investigation*, vol. 40, 2022, 301330.
- [23] Küchler, A., Mantovani, A., Han, Y., Bilge, L., and Balzarotti, D. "Does Every Second Count? Time-based Evolution of Malware Behavior in Sandboxes." *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2021.
- [24] Onwuegbuche, F. C., Olaoluwa, A., Jurecut, A. D., and Pasquale, L. "MLRan: A Behavioural Dataset for Ransomware Analysis and Detection." *arXiv preprint arXiv:2505.18613*, 2025.
- [25] Botacin, M., Ceschin, F., Sun, R., Oliveira, D., and Grégio, A. "Challenges and pitfalls in malware research." *Computers & Security*, vol. 106, 2021, 102287.
- [26] Putrevu, M. A., Putrevu, V. S. C., and Shukla, S. K. "HiPeR: Early Detection of a Ransomware Attack using Hardware Performance Counters." *Digital Threats: Research and Practice*, vol. 4, no. 3, 2023, article 43, pp. 1–24.
- [27] Pendlebury, F., Pierazzi, F., Jordaney, R., Kinder, J., and Cavallaro, L. "TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time." *Proceedings of the 28th USENIX Security Symposium*, 2019, pp. 729–746.

- [28] Arp, D., Quiring, E., Pendlebury, F., Warnecke, A., Pierazzi, F., Wressnegger, C., Cavallaro, L., and Rieck, K. "Dos and Don'ts of Machine Learning in Computer Security." Proceedings of the 31st USENIX Security Symposium, 2022, pp. 3971–3988.
- [29] Barbero, F., Pendlebury, F., Pierazzi, F., and Cavallaro, L. "Transcending TRANSCEND: Revisiting Malware Classification in the Presence of Concept Drift." Proceedings of the 2022 IEEE Symposium on Security and Privacy (S&P), 2022, pp. 805–823.
- [30] Ceschin, F., Botacin, M., Gomes, H. M., Pinage, F., Oliveira, L. S., and Grégio, A. "Fast & Furious: On the modelling of malware detection as an evolving data stream." Expert Systems with Applications, vol. 212, 2023, 118590.
- [31] Sommer, R., and Paxson, V. "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection." Proceedings of the 2010 IEEE Symposium on Security and Privacy (S&P), 2010, pp. 305–316.
- [32] Sahai, A. "Bypassing EDR: How Akira Used an Unsecured Webcam to Deploy Ransomware." Ampcus Cyber ShadowOpsIntel, March 2025. Reporting incident response findings by S-RM.
- [33] Ineza, Y., Jackson, G., Niyonkuru, P., Kevil, J., and Serwadda, A. "Intermittent File Encryption in Ransomware: Measurement, Modeling, and Detection." arXiv preprint arXiv:2510.15133, 2025.
- [34] Microsoft Corporation. "[MS-SMB2]: Server Message Block (SMB) Protocol Versions 2 and 3." Microsoft Open Specifications, 2023.
- [35] Chen, L., Yang, C.-Y., Paul, A., and Sahita, R. "Towards Resilient Machine Learning for Ransomware Detection." arXiv preprint arXiv:1812.09400, 2018 (revised 2019).